

Rapport — Laboratoire 06

Question 1

Lequel de ces fichiers Python représente la logique de la machine à états décrite dans les diagrammes du document arc42? Est-ce que son implémentation est complète ou y a-t-il des éléments qui manquent? Illustrez votre réponse avec des extraits de code.

Le fichier `src/controllers/order_saga_controller.py` représente la logique de la machine à états. Dans la méthode `run()`, le contrôleur vérifie l'état courant de la saga et appelle le handler correspondant. Par exemple :

```
elif self.current_saga_state == OrderSagaState.ORDER_CREATED:
    self.decrease_stock_handler = DecreaseStockHandler(
        self.create_order_handler.order_id,
        order_data['items']
    )
    self.current_saga_state = self.decrease_stock_handler.run()
```

L'implémentation n'est toutefois pas complète. Lorsque la saga atteint l'état `STOCK_INCREASED`, le contrôleur passe directement à l'état `ORDER_DELETED` sans appeler `DeleteOrderHandler` :

```
elif self.current_saga_state == OrderSagaState.STOCK_INCREASED:
    self.logger.debug(
        "TODO: implémentez et utilisez la classe DeleteOrderHandler "
        "et ensuite changez à l'état ORDER_DELETED"
    )
    self.current_saga_state = OrderSagaState.ORDER_DELETED
```

Il manque donc l'importation, la création et l'appel du `DeleteOrderHandler`, qui doit réellement supprimer la commande avant de retourner l'état `ORDER_DELETED`.

Question 2

Est-ce que le handler `CreateOrderHandler` connecte à une base de données directement pour créer des commandes? Illustrez votre réponse avec des extraits de code.

Non, le handler `CreateOrderHandler` ne se connecte pas directement à une base de données. Il envoie une requête HTTP au service Store Manager en passant par l'API Gateway :

```
response = requests.post(
    f'{config.API_GATEWAY_URL}/store-manager-api/orders',
    json=self.order_data,
```

```
headers={'Content-Type': 'application/json'}  
)
```

Les informations de la commande sont envoyées au format JSON. C'est le Store Manager qui s'occupe de créer et d'enregistrer la commande dans sa base de données. Lorsque la requête réussit, le handler récupère seulement l'identifiant de la commande retourné par le service :

```
if response.ok:  
    data = response.json()  
    self.order_id = data['order_id'] if data else 0
```

Ainsi, `CreateOrderHandler` communique avec un autre microservice par HTTP et ne contient aucun code de connexion directe à une base de données.

Question 3

Quelle requête dans la collection Postman du Labo 05 correspond à l'endpoint appelé par `CreateOrderHandler`? Illustrez votre réponse avec des captures d'écran ou extraits de code.

La requête correspondante dans la collection Postman du Labo 05 est celle qui permet de créer une commande :

```
POST http://localhost:8080/store-manager-api/orders
```

Elle contient les informations de l'utilisateur et des articles commandés :

```
{  
  "user_id": 1,  
  "items": [  
    {  
      "product_id": 3,  
      "quantity": 2  
    },  
    {  
      "product_id": 2,  
      "quantity": 4  
    }  
  ]  
}
```

Cette requête correspond à l'appel HTTP effectué dans `CreateOrderHandler` :

```
response = requests.post(  
    f'{config.API_GATEWAY_URL}/store-manager-api/orders',
```

```
    json=self.order_data,  
    headers={'Content-Type': 'application/json'}  
)
```

Dans Postman, l'adresse `localhost:8080` est utilisée, car Postman s'exécute à l'extérieur du réseau Docker. Le handler utilise plutôt l'adresse interne de l'API Gateway définie dans `config.API_GATEWAY_URL`.

Capture d'écran à ajouter : la requête Postman montrant la méthode `POST`, l'URL, le corps JSON et la réponse contenant l'identifiant `order_id`.

Question 4

Quel endpoint avez-vous appelé pour modifier le stock? Quelles informations de la commande avez-vous utilisées? Illustrez votre réponse avec des extraits de code.

Pour modifier le stock, j'ai appelé l'endpoint suivant du Store Manager en passant par l'API Gateway :

```
PUT /store-manager-api/stocks
```

J'ai utilisé la liste des articles de la commande contenue dans `self.order_item_data`. Chaque article comprend son identifiant `product_id` et la quantité commandée `quantity`. L'opération `"-"` indique qu'il faut diminuer les quantités en stock.

```
response = requests.put(  
    f'{config.API_GATEWAY_URL}/store-manager-api/stocks',  
    json={  
        "items": self.order_item_data,  
        "operation": "-"  
    },  
    headers={'Content-Type': 'application/json'}  
)
```

Par exemple, les données envoyées peuvent avoir la forme suivante :

```
{  
  "items": [  
    {  
      "product_id": 3,  
      "quantity": 2  
    },  
    {  
      "product_id": 2,  
      "quantity": 4  
    }  
  ],  
}
```

```
"operation": "-"  
}
```

Question 5

Quel endpoint avez-vous appelé pour générer une transaction de paiement? Quelles informations de la commande avez-vous utilisées? Illustrez votre réponse avec des extraits de code.

Pour générer une transaction de paiement, j'ai appelé l'endpoint suivant de l'API de paiement en passant par l'API Gateway :

```
POST /payments-api/payments
```

Avant de créer le paiement, j'ai récupéré les informations de la commande avec son identifiant :

```
GET /store-manager-api/orders/{order_id}
```

Cette requête permet d'obtenir le montant total calculé par le Store Manager :

```
response = requests.get(  
    f'{config.API_GATEWAY_URL}/store-manager-api/orders/{self.order_id}'  
)  
  
if response.ok:  
    order_info = response.json()  
    self.total_amount = order_info.get('total_amount', 0)
```

Pour créer la transaction, j'ai utilisé l'identifiant de la commande ainsi que son montant total :

```
response = requests.post(  
    f'{config.API_GATEWAY_URL}/payments-api/payments',  
    json={  
        "order_id": self.order_id,  
        "amount": self.total_amount  
    },  
    headers={'Content-Type': 'application/json'})
```

Le montant n'est pas recalculé par l'orchestrateur : il est récupéré auprès du Store Manager, qui constitue la source des informations de la commande.

Question 6

Quelle est la différence entre appeler l'orchestrateur Saga et appeler directement les endpoints des services individuels? Quels sont les avantages et inconvénients de chaque approche? Illustrez votre réponse avec des captures d'écran ou extraits de code.

Lorsque le client appelle l'orchestrateur Saga, il envoie une seule requête :

```
POST /saga/order
```

```
{
  "user_id": 1,
  "items": [
    {
      "product_id": 3,
      "quantity": 2
    },
    {
      "product_id": 2,
      "quantity": 4
    }
  ]
}
```

L'orchestrateur coordonne ensuite les différentes opérations dans le bon ordre : création de la commande, diminution du stock et création du paiement. En cas d'échec, il déclenche les opérations de compensation nécessaires, comme la restauration du stock ou la suppression de la commande.

```
if self.current_saga_state == OrderSagaState.START:
    self.current_saga_state = self.create_order_handler.run()
elif self.current_saga_state == OrderSagaState.ORDER_CREATED:
    self.current_saga_state = self.decrease_stock_handler.run()
elif self.current_saga_state == OrderSagaState.STOCK_DECREASED:
    self.current_saga_state = self.create_payment_handler.run()
```

Avec des appels directs, le client doit appeler lui-même chaque service :

```
POST /store-manager-api/orders
PUT /store-manager-api/stocks
GET /store-manager-api/orders/{order_id}
POST /payments-api/payments
```

Le client doit également gérer l'ordre des opérations et effectuer les compensations lorsqu'une requête échoue.

Approche	Avantages	Inconvénients
Orchestrateur Saga	Un seul endpoint pour le client, coordination centralisée, compensations automatiques et meilleure cohérence des données	Ajout d'un service critique, complexité de l'orchestrateur et risque de point de défaillance unique
Appels directs	Simplicité pour une opération isolée, contrôle direct des services et absence d'orchestrateur central	Client plus complexe, logique dupliquée et risque d'incohérence lorsqu'une opération réussit et qu'une autre échoue

L'orchestrateur Saga est donc mieux adapté à la création complète d'une commande, car celle-ci nécessite plusieurs opérations dépendantes. Les appels directs restent utiles pour tester ou exécuter une opération individuelle.

Captures d'écran à ajouter :

1. la réponse de l'appel **POST /saga/order** dans Postman;
2. les transitions d'état dans les logs Docker;
3. la trace distribuée complète dans Jaeger.